

Working with Data

Session 3 — Can I trust the answer?

Block 6 — Validations as code, and the note that goes with the number

This block in one sentence

A check that cannot fail is not a check.

A pipeline is finished when it runs from a fresh clone, refuses bad data out loud, and its number arrives with the sentence that says what it means.

Nothing is typed in this lecture. On a prediction slide you write your answer and one reason, in silence; we vote; then the next slide shows what it printed, on a finished Lab 5 pipeline.

What a validation is

An assertion about the data that **stops the run** when it is false.

```
dupes = con.sql("SELECT COUNT(*) - COUNT(DISTINCT (geo, year, wst_oper)) FROM silver").fetchone()[0]
assert dupes == 0, f"{dupes} duplicate (geo, year, wst_oper) in silver"
```

- Not a `print`: nobody reads the log of a run that succeeded
- Not a comment: `# order_id is unique` checks nothing
- **The message carries the number:** `<n> duplicate (geo, year, wst_oper) in silver` says what and how much

Where the checks run

the Block 5 figure, with the gates filled in: scripts/checks.py



every check writes one record to output/checks_report.json; any "stop" stops the pipeline

Family 1 — key

The key of the stage is unique, and never **NULL**.

```
SELECT COUNT(*) - COUNT(DISTINCT (geo, year, wst_oper)) AS duplicates,  
       COUNT(*) FILTER (WHERE geo IS NULL OR year IS NULL OR wst_oper IS NULL) AS null_keys  
FROM 'data/silver/waste.parquet'
```

Written against **the key of the stage it checks** (Block 5): silver's key, not gold's.

Watch — the key check, on the Lab 5 silver

cell 15

```
con.sql("""
    SELECT COUNT(*)                                AS n_rows,
           COUNT(DISTINCT (geo, year, wst_oper)) AS n_keys,
           COUNT(DISTINCT (geo, year))          AS n_geo_year
    FROM 'data/silver/waste.parquet'
""").df()
```

	n_rows	n_keys	n_geo_year
0	5565	5565	570

Silver's key, and a key that is not silver's, side by side.

Family 2 — domain

Every value is one the column is allowed to hold.

rule	from	Lab 5 silver
allowed values	a census	<code>wst_oper</code> : the 10 codes the census finds, all in <code>odelist_wst_oper.tsv</code> ; <code>flag</code> : in the flag list
ranges	the meaning	kilograms per inhabitant ≥ 0
one unit	the dictionary	only <code>KG_HAB</code>
inside the window	the extract	<code>year</code> from 2010 to 2024

A row that breaks a domain rule is **quarantined**: `rejects.csv` , with the reason. The run goes on.

Family 3 — count

Every row is accounted for.

- Rows in = rows out + rejected (+ excluded on purpose, by name). Lab 5's accounting line is a count check
- A join did not change the left table's row count (Session 2's discipline, as code)
- No period is missing from a series: `GROUP BY` shows only groups that exist, **so a gap is silent**

```
SELECT year, COUNT(value) AS countries_with_a_value
FROM 'data/silver/waste.parquet'
WHERE wst_oper = 'GEN' AND NOT is_aggregate
GROUP BY year ORDER BY year          -- a year with no rows would not appear at all
```

Family 4 — reconciliation: the identity the metric permits

Municipal waste generated, 2022, the EU's 27 members, from the waste file:

metric	identity	EU27 row	from the 27 members
thousand tonnes — additive	the members sum to the total	229,690	sum: 229,687 (rounding)
kg per inhabitant — a ratio	a sum is meaningless	514	sum: 14,243 ×
	a plain average is wrong	514	average: 527.5 ×
	weighted by population (each member's tonnes ÷ its kg per person)	514	weighted: 513.9 ✓

Two computations, independent, that must agree — by the identity this metric allows, to its stated rounding.

Compare sums with a tolerance, never `==`

From the Block 5 emissions file, 2022, total greenhouse gases, million tonnes:

```
eu_row, members = 3359.59802, 3359.59800000000004    # the EU27 row; the 27 members, summed by DuckDB
eu_row == members                                     # False
assert abs(eu_row - members) <= 0.0002, f"EU27 row and its members differ by {eu_row - members:.5f} Mt"
```

- The source prints 5 decimals: 27 rounded members can be off by up to $27 \times 0.0000005 = 0.000135$ Mt
- A sum of floats differs in its last digits. Money: to the cent, or `ROUND` both sides
- **The tolerance comes from the rounding you can name. The difference goes in the message.**

Predict, then watch — a key check that cannot see a duplicate

```
con.sql("""  
    SELECT COUNT(*) AS rows_with_no_geo  
    FROM 'data/silver/waste.parquet'  
    WHERE geo IS NULL  
""").df()
```

It is called a key check. Predict: does it pass on the Lab 5 silver? And, in one sentence: **what wrong silver would it also pass?**

A key check that cannot see a duplicate — what it printed

```
cell 16 con.sql("""  
    SELECT COUNT(*) AS rows_with_no_geo  
    FROM 'data/silver/waste.parquet'  
    WHERE geo IS NULL  
    """).df()
```

	rows_with_no_geo
0	0

The gallery: checks that pass on wrong data

kind	example	the wrong data it would pass
incorrect	the key check that tests <code>geo IS NULL</code>	every row twice
weak	<code>assert n > 0</code>	a silver with one row in it. It fails only on an empty table
tautological	<code>assert total == total</code> ; counting a <code>CASE</code> 's own labels; a <code>try/except</code> that swallows	anything at all
missing	no count check	a filter that drops a third of the rows

For every check: **name the wrong data it would pass**. Not "make it fail": a weak check can be made to fail, and is still weak.

Predict, then watch — a coverage check that forgot its parentheses

Every place in the Lab 5 silver should have a 2024 figure for waste generated or treated (**GEN** or **TRT**). The check passes when the two numbers are equal:

```
con.sql("""
    SELECT (SELECT COUNT(DISTINCT geo) FROM 'data/silver/waste.parquet') AS places,
           COUNT(DISTINCT geo) AS with_a_2024_figure
    FROM 'data/silver/waste.parquet'
    WHERE year = 2024 AND value IS NOT NULL AND wst_oper = 'GEN' OR wst_oper = 'TRT'
""").df()
```

Predict: does the check pass? Read the **WHERE** the way SQL reads it.

A coverage check that forgot its parentheses — what it printed

cell 16b

```
con.sql("""
    SELECT (SELECT COUNT(DISTINCT geo) FROM 'data/silver/waste.parquet') AS places,
           COUNT(DISTINCT geo) AS with_a_2024_figure
    FROM 'data/silver/waste.parquet'
    WHERE year = 2024 AND value IS NOT NULL AND wst_oper = 'GEN' OR wst_oper = 'TRT'
""").df()
```

	places	with_a_2024_figure
0	38	38

The same check, with the parentheses

```
cell 16c con.sql("""  
    SELECT (SELECT COUNT(DISTINCT geo) FROM 'data/silver/waste.parquet') AS places,  
           COUNT(DISTINCT geo) AS with_a_2024_figure  
    FROM 'data/silver/waste.parquet'  
    WHERE year = 2024 AND value IS NOT NULL AND (wst_oper = 'GEN' OR wst_oper = 'TRT')  
""").df()
```

	places	with_a_2024_figure
0	38	26

AND binds tighter than **OR**. Parentheses every time you mix them — or `wst_oper IN ('GEN', 'TRT')`.

Predict, then watch — rows, and values

```
con.sql("""  
    SELECT COUNT(*) AS n_rows, COUNT(value) AS n_values  
    FROM 'data/silver/waste.parquet'  
""").df()
```

Two silvers: the colleague's, from the Lab 5 starter as cloned, and a finished one. Predict: the same number of rows? The same number of values? Why?

The query is cell 17 of the lecture notebook: run it in the starter and in your finished Lab 5.

Rows, and values — what it printed

cell 17

```
con.sql("""
    SELECT COUNT(*) AS n_rows, COUNT(value) AS n_values
    FROM 'data/silver/waste.parquet'
    """).df()
```

	n_rows	n_values
0	5565	4828

a finished Lab 5

cell 17

```
con.sql("""
    SELECT COUNT(*) AS n_rows, COUNT(value) AS n_values
    FROM 'data/silver/waste.parquet'
    """).df()
```

	n_rows	n_values
0	5565	4022

the Lab 5 starter, as cloned: the colleague's silver

A row you reject, a run you stop

	what is wrong	what happens	families
quarantine	one row : an unparseable value, a date outside the window, a negative price	set aside in <code>rejects.csv</code> with the reason, counted , the run goes on	domain
stop	an invariant : a duplicate key, a sum that does not match its total, rows that vanished, more rejects than the stated tolerance	the run stops, the number in the message , nothing downstream uses this run's gold	key, count, reconciliation

The pipeline says which it did, in its output.

Fail fast, and idempotency

Fail fast and loud. The run stops at the stage that failed, with the number in the message. `pipeline.py` exits with an error, so nothing after it runs.

Idempotency: run twice, same tables. And a check that proves it:

```
before = {f: sha256(f.read_bytes()).hexdigest() for f in written_files}
# ... run clean.py and report.py again ...
after  = {f: sha256(f.read_bytes()).hexdigest() for f in written_files}
assert before == after, f"{sum(before[f] != after[f] for f in before)} files changed on a second run"
```

What breaks it: appending instead of overwriting; the time of the run written into a table; a row order SQL does not promise without `ORDER BY`.

Watch — the Lab 5 pipeline gains three checks

```
$ uv run python pipeline.py | tail -n 7
== scripts/checks.py
pass      key          silver_key          observed 0    (0 duplicate (geo, year, wst_oper))
pass      domain       silver_values_valid observed 0    (0 negative values or unknown flags in silver)
pass      count        silver_accounted    observed 0    (raw = silver + rejected, gap 0)
3 checks, none says stop

== pipeline finished
```

A **throwaway copy** of a finished Lab 5 (never your own `data/raw/`), with `scripts/checks.py` : key, domain, count.

Predict, then watch — one real row, twice

```
grep 'A,GEN,KG_HAB,DE' data/raw/env_wasmun.tsv > de_row.tsv    # one real row: Germany  
cat de_row.tsv >> data/raw/env_wasmun.tsv                    # ... now in the file twice  
uv run python pipeline.py
```

Predict: **which** of the three checks stops the run, and what **number** does its message carry?

One real row, twice — what it printed

```
$ grep 'A,GEN,KG_HAB,DE' data/raw/env_wasmun.tsv > de_row.tsv
$ cat de_row.tsv >> data/raw/env_wasmun.tsv
$ uv run python pipeline.py | tail -n 7
== scripts/checks.py
stop      key          silver_key          observed 15    (0 duplicate (geo, year, wst_oper))
pass      domain         silver_values_valid observed 0    (0 negative values or unknown flags in silver)
pass      count          silver_accounted    observed 0    (raw = silver + rejected, gap 0)
STOP: silver_key observed 15, wanted 0 duplicate (geo, year, wst_oper)

== pipeline STOPPED at scripts/checks.py. Nothing after it ran.
```

Predict, then watch — the check is annoying, so it goes

The duplicated row stays in the raw file. In `scripts/checks.py` :

```
CHECKS = [silver_key, silver_values_valid, silver_accounted]    # before  
CHECKS = [silver_values_valid, silver_accounted]                # after
```

Predict: does the run pass now? If it passes, what does gold say about **Germany**?

The check is annoying, so it goes — what it printed

```
$ grep '^CHECKS' scripts/checks.py
CHECKS = [silver_values_valid, silver_accounted]
$ uv run python pipeline.py | tail -n 6
== scripts/checks.py
pass      domain      silver_values_valid  observed 0    (0 negative values or unknown flags in silver)
pass      count       silver_accounted     observed 0    (raw = silver + rejected, gap 0)
2 checks, none says stop

== pipeline finished
$ grep -E '^(geo|DE),' data/gold/waste_per_inhabitant.csv
geo,kg_2010,flag_2010,kg_2024,flag_2024,change_kg
DE,1204.0,,1256.0,e,52.0
```

silver_key taken out of CHECKS in scripts/checks.py; the duplicated German row is still in the raw file

The demo, as it ran: lecture notebook, section 18.

The fresh-clone test

```
git clone <the repo> fresh-copy  
cd fresh-copy  
uv sync  
uv run python pipeline.py
```

DS1's *Restart Kernel and Run All*, grown up. It proves the pipeline needs nothing that lives only on your laptop: no file you forgot to commit, no silver left over from yesterday.

It is Homework 3's execution test. Run it before you submit, and say in the note that you did.

The note that goes with the number

Five to eight sentences, for a reader who does not know SQL and will make a decision:

1. **The metric** — its grain, its filter, its unit
2. **What was excluded**, and how many
3. **The assumptions**
4. **The reconciliation**, stated: which two numbers agree
5. **What the data cannot answer** — a claim about the data: *"this cannot tell you X, because the data has no Y"*

Not a hedge. "More research is needed" is not a claim; "the file has a break flag but no description of the break" is.

A note, for Lab 5's gold

Municipal waste generated per person in 2024, in kilograms, by country, from Eurostat's `env_wasmun`. 25 of the 37 countries in the file report a 2024 value; 12 do not, among them Austria, Italy and Ireland. 10 of the 25 values carry a Eurostat flag — 4 estimated, 3 provisional, 1 both, 1 imputed, 1 a break in the series — and the table shows the flag beside each. The EU as a whole, 517 kg (imputed by Eurostat), is reported apart from the countries. All 11,130 observations in the file are accounted for: none rejected, 5,565 in thousand tonnes excluded, 135 for the EU, 5,430 kept. This cannot tell you whether Sweden's figure changed because of what people threw away or because of how it was counted, because the file marks a break in the series but does not describe it.

Two appendices a skeptic reads

The data dictionary — every silver column: type, meaning, unit, allowed values, what missing means, what was done to it (Lab 5's `docs/dictionary.md`).

The reconciliation table — every gold figure beside its independent computation:

gold figure	independent computation	identity	agrees?
EU27 waste, 2022, thousand tonnes	sum of the 27 members	sum	229,690 vs 229,687: rounding
EU27 waste per person, 2022	members weighted by population	weighted mean	514 vs 513.9

Git thread — clean means committed and ignored

```
git status          # nothing to commit, working tree clean
```

- **Clean** means: raw data and scripts committed, generated files ignored
- `data/silver/` , `data/gold/` , `output/` never appear in it
- The fresh-clone test runs from a clean state. Run it, then say so in your note

AI thread — can you make it fail?

Ask an assistant to write the checks for your pipeline. Then, for **each** check it writes:

1. What data would make this check fail? Ask it, then build that data **in a copy**
2. Run the pipeline on the copy. Did it fail, with the number?
3. What wrong data would it pass? If the answer is "a lot", the check is weak

A check the assistant cannot make fail on purpose is not a check. Neither is one you cannot.

5-minute stand-and-stretch — start the clone first

In the **terminal** (Git Bash on Windows, Terminal on macOS), in your course folder, then stand up:

```
git clone https://github.com/earino/ecbs5294-lab06-validations.git  
cd ecbs5294-lab06-validations  
uv sync
```

It downloads while you stretch. Water, window, hallway. Back in five minutes, then **Lab 6** — longer than usual today: the lecture gives its spare minutes to the lab.

Lab 6 — The pipeline that passes its own tests

First minute: check the clone you started ran — VS Code → **File** → **Open Folder...** → `ecbs5294-lab06-validations` .

Olist orders; `pipeline.py` runs `clean.py` , `report.py` , `checks.py` . The run ends with "**every check passed**". The report says the shop took **2,555 orders**. `data/raw/orders.csv` has one row per order: count them.

1. `uv run python pipeline.py` — read each check's record
2. For each of the three checks: **the wrong data it would pass**. The missing orders: `notebooks/diagnose.ipynb`
3. Key, count (with idempotency), reconciliation in `checks.py` ; watch it refuse
4. The decision
5. **The domain family**, minutes 22–32 (the lab starts ~15 minutes early)
6. The note and `DIAGNOSIS.md` : minute 32, 37 at the latest

By minute 22: a run that refuses on your count check (idempotency can wait). Else: hand up.

The last ten minutes — explain it to a neighbour

Speaker, about a minute, pointing at the screen, not reading the note: symptom → cause → evidence → change → verification.

Listener asks:

1. Show me the query that proves it.
2. Why was it wrong, not just where?
3. You added `assert rows_out == rows_in`. Next month's file has three genuinely invalid rows that the clean step is right to reject. What happens, and what should the check be?

Swap. Unsure, or you disagree? **Hand up**. Then the answer, for everyone. Before you leave: `DIAGNOSIS.md` and `NOTE.md` to the Lab 6 checkpoint on Moodle.